

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a2fba3cd88661db7b.jsonl`  
- SHA-256: `b9417a1a683dd91a6ea3796653ce5644078ad978dc8097caf77ec0e7ce46c493`  
- Source modified: `2026-07-02T03:02:02+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T03:00:07.161Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

```
{  
  "title": "Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ?  
no cancellation, so the L4 keeps billing and its late result is never ingested",  
  "category": "critical-fix",  
  "severity": "P2",  
  "file": "backend/compute/cloud_run.py",  
  "line": 173,  
  "evidence": "cloud_run.py:173-178: run_infer blocks on poll_until(...);  
execution_policy.py:183-189 raises PolicyTimeout after max_wait_sec (default 1800s). The  
exception propagates to main.py:1068-1070 which marks the job failed (`cloud dispatch error`) ?  
no code anywhere in cloud_run.py calls executions.cancel/delete (grep: no cancel API), and the  
late-arriving done-marker/intervals.csv are never ingested (the local job is already  
terminal).",  
  "why_it_matters": "Direct GPU cost leak plus wasted results on the cloud-serving path (owner-  
gated go-live area, but the code path is live behind compute='cloud').",  
  "proposed_fix": "On PolicyTimeout, best-effort cancel the execution via the Jobs API  
(executions.cancel with the stored execution name) and log the execution id in the failure  
message; optionally check the done-marker once more before failing.",  
  "effort": "M",  
  "tracked_in": ""  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer,
C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-
data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator,
C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field;
non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup',
C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited
code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent
commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete
failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it
owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training,
product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only
commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you
personally saw the defect/debt in current code; if you cannot reproduce the claim from the code,
say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T03:00:15.435Z)

Already up to date.

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)

3. user (2026-07-02T03:00:15.440Z)

```
1      """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2      for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4      Flow (the cloud-serving compute path):
5      1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6          --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7      2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8          completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9          it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10         cold-start is absorbed by the async poll.
11      3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12         ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19      owner-verified on the M7 real run.
20      """
21
22      from __future__ import annotations
23
24      import json
25      import logging
26      import os
27      import tempfile
28      import uuid
29      from abc import ABC, abstractmethod
30      from typing import Any, List, Optional
31
32      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33      from backend.infrastructure.execution_policy import (
34          DEFAULT_POLICY,
35          ExecutionPolicy,
36          poll_until,
37      )
38      from backend.storage import fetch, get_storage, parse_uri
39
40      LOG = logging.getLogger("compute.cloud_run")
41
42      # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
43      # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
```

```

runner.)
44     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
45         "deployment.compute_target=local_gpu",
46         "indexing.detector_impl=native",
47     )
48
49
50     class CloudRunJobClient(ABC):
51         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
52
53         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
54         overrides the job's container ``args`` for this execution and starts it."""
55
56         @abstractmethod
57         def run_job(
58             self,
59             job_name: str,
60             argv: List[str],
61             *,
62             region: str,
63             project: Optional[str] = None,
64         ) -> str:
65             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
66             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
67
68
69     class GoogleCloudRunJobClient(CloudRunJobClient):
70         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
71         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
72
73         def run_job(
74             self,
75             job_name: str,
76             argv: List[str],
77             *,
78             region: str,
79             project: Optional[str] = None,
80         ) -> str:
81             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
82             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
83             # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
84             if not project:
85                 raise RuntimeError(
86                     "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
87                     "to resolve the job path; set it on the control plane (see
get_compute_backend)."
88                 )
89             try:
90                 from google.cloud import run_v2 # type: ignore[attr-defined]
91             except Exception as exc: # pragma: no cover - real SDK not present in CI
92                 raise RuntimeError(
93                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
94                     "control plane (it is intentionally NOT a CI/test dependency)."
95                 ) from exc

```

```

96         client = (
97             run_v2.JobsClient()
98         ) # pragma: no cover - exercised only on the real M7 run
99         name = client.job_path(project, region, job_name)
100         overrides = run_v2.RunJobRequest.Overrides(
101             container_overrides=[
102                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
103             ]
104         )
105         op = client.run_job(
106             request=run_v2.RunJobRequest(name=name, overrides=overrides)
107         )
108         return (
109             getattr(op, "metadata", None)
110             and getattr(op.metadata, "name", "")
111             or "execution"
112         )
113
114
115 class CloudRunCompute(ComputeBackend):
116     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
117
118     def __init__(
119         self,
120         *,
121         run_client: CloudRunJobClient,
122         job_name: str,
123         region: str,
124         project: Optional[str] = None,
125         storage_config: Optional[dict] = None,
126         policy: ExecutionPolicy = DEFAULT_POLICY,
127         token: Optional[str] = None,
128         container_overrides: Optional[tuple[str, ...]] = None,
129     ) -> None:
130         self._client = run_client
131         self._job_name = job_name
132         self._region = region
133         self._project = project
134         self._storage_config = storage_config or {}
135         self._policy = policy
136         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
137         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
138         self._token = token
139         self._container_overrides = (
140             _DEFAULT_CONTAINER_OVERRIDES
141             if container_overrides is None
142             else tuple(container_overrides)
143         )
144
145     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
146         out_root = spec.out_uri.rstrip("/")
147         token = self._token or uuid.uuid4().hex
148         done_uri = f"{out_root}/_dispatch/{token}.done"
149         argv: List[str] = [
150             "infer",
151             "--video-uri",
152             spec.video_uri,
153             "--out-uri",
154             spec.out_uri,
155             "--done-uri",
156             done_uri,
157         ]
158         if spec.segmenter:

```

```

159         argv += ["--segmenter", spec.segmenter]
160     for kv in (*spec.config_overrides, *self._container_overrides):
161         argv += ["--set", kv]
162
163     execution = self._client.run_job(
164         self._job_name, argv, region=self._region, project=self._project
165     )
166     LOG.info(
167         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
168         self._job_name,
169         execution,
170         done_uri,
171     )
172
173     marker = poll_until(
174         lambda: self._read_marker(done_uri),
175         self._policy,
176         label="cloud-run-infer",
177         logger=LOG,
178     )
179     video_id = str(marker.get("video_id", ""))
180     # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
181     # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
182     rc = int(marker.get("returncode", 1))
183     if not video_id:
184         LOG.warning(
185             "cloud-run: job=%s completion marker present but empty/unreadable (no "
186             "video_id) ? treating as failure",
187             self._job_name,
188         )
189     rc = rc or 1
190     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
191     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
192     LOG.info(
193         "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
194     )
195     return ComputeResult(
196         returncode=rc,
197         outputs_uri=outputs_uri,
198         video_id=video_id,
199         report=report,
200         execution=str(execution or ""),
201     )
202
203     # --- storage-backed completion signal (bucket-poll) ---
204     def _read_marker(self, done_uri: str) -> Optional[dict]:
205         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
206         ref = parse_uri(done_uri)
207         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
208             return None
209         return (
210             self._read_json(done_uri) or {}
211         ) # empty dict still terminates the poll (present == done)
212
213     def _read_json(self, uri: str) -> dict:
214         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
215         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
216         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
217         try:

```

```

218         ref = parse_uri(uri)
219         dst = os.path.join(
220             tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
221         )
222         local = fetch(uri, dst, config=self._storage_config)
223         with open(local, "r", encoding="utf-8") as f:
224             data: Any = json.load(f)
225         return data if isinstance(data, dict) else {}
226     except Exception:
227         LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
228         return {}
229

```

4. user (2026-07-02T03:00:22.848Z)

```

140         attempts,
141         policy.op_timeout_sec,
142     )
143     if attempt < attempts - 1:
144         wait = _backoff_sec(policy, attempt)
145         log.debug("%s: backing off %.1fs before retry", label, wait)
146         sleep(wait)
147     assert last_exc is not None
148     log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
149     raise last_exc
150
151
152     def poll_until(
153         check: Callable[[], Optional[T]],
154         policy: ExecutionPolicy = DEFAULT_POLICY,
155         *,
156         label: str = "op",
157         logger: Optional[logging.Logger] = None,
158         sleep: Callable[[float], None] = _time.sleep,
159         monotonic: Callable[[], float] = _time.monotonic,
160     ) -> T:
161         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
162         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
163
164         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
165         """
166         log = logger or LOG
167         start = monotonic()
168         polls = 0
169         log.info(
170             "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
171             label,
172             policy.poll_every_n_sec,
173             policy.max_wait_sec,
174         )
175         while True:
176             result = check()
177             if result is not None:
178                 log.info(
179                     "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
180                 )
181                 return result
182             elapsed = monotonic() - start
183             if elapsed >= policy.max_wait_sec:
184                 log.warning(
185                     "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
186                 )

```

```

187         raise PolicyTimeout(
188             f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
189         )
190     polls += 1
191     log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
192     sleep(policy.poll_every_n_sec)
193

```

5. user (2026-07-02T03:00:24.585Z)

```

b6214e5 chore: ruff format the whole repo (259 files) (#387)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
688d28f fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT (#342)
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)
(#287)

```

6. user (2026-07-02T03:00:25.810Z)

```

badminton-highlight-indexer\backend\ui\assets\index-ukuS9wxa.js:17:[Omitted long matching line]
badminton-highlight-indexer\backend\ui\assets\index-ukuS9wxa.js:25:[Omitted long matching line]
badminton-highlight-indexer\backend\ui\assets\index-ukuS9wxa.js:48:[Omitted long matching line]
badminton-highlight-indexer\backend\ui\assets\index-ukuS9wxa.js:49:[Omitted long matching line]
badminton-highlight-indexer\backend\ui\assets\index-ukuS9wxa.js:60:[Omitted long matching line]

```

7. user (2026-07-02T03:00:32.373Z)

```

1020
1021     storage_cfg = getattr(cfg, "storage", None)
1022
1023     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
1024     try:
1025         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
1026     except ValueError as e:
1027         return _failed(
1028             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
1029         )
1030     if input_ref.backend == "local":
1031         return _failed(
1032             "cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
1033             "can't be read by the Cloud Run job. Put the video in your bucket first.",
1034             False,
1035         )
1036     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1037     if not out_root:
1038         return _failed(
1039             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
1040             "? that's where the Cloud Run job writes the result.",
1041             False,
1042         )
1043
1044     notify("dispatching (cloud_run)")
1045     logger.info(
1046         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
1047         req.video_path,
1048         out_root,
1049     )
1050     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
1051     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key

```

```

needed); without
1052     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
1053     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
1054     overrides = []
1055     sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
1056     if sk is not None:
1057         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
1058     spec = ComputeJobSpec(
1059         video_uri=req.video_path,
1060         out_uri=out_root,
1061         segmenter=seg_name,
1062         config_overrides=tuple(overrides),
1063     )
1064     try:
1065         result = compute.run_infer(
1066             spec
1067         ) # dispatch the Cloud Run Job + bucket-poll to completion
1068     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1069         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
1070         return _failed(f"cloud dispatch error: {e}", True)
1071     if result.returncode != 0:
1072         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
1073     if result.video_id and result.video_id != video_id:
1074         # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
1075         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
1076         logger.warning(
1077             "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
1078             "and the worker hash; ingesting under the local id.",
1079             result.video_id,
1080             video_id,
1081         )
1082
1083     notify("ingesting (cloud_run)")
1084     storage_dict = (
1085         storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
1086     )
1087     try:
1088         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
1089     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows

```

8. user (2026-07-02T03:00:33.902Z)

```

8: ``max_wait_sec`` deadline).
42:     """An op hung past ``op_timeout_sec`` or never became ready within ``max_wait_sec``."""
46: class ExecutionPolicy:
54:     max_wait_sec: float = 1800.0 # total wall budget for a poll loop
75: DEFAULT_POLICY = ExecutionPolicy()
162:     ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
173:         policy.max_wait_sec,
183:         if elapsed >= policy.max_wait_sec:
188:             f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"

```

9. assistant (2026-07-02T03:00:52.123Z)

Now let me check how the backend is constructed (policy used) and whether the Cloud Run job deployment sets its own task timeout, which affects the "keeps billing" claim.

10. user (2026-07-02T03:00:54.954Z)

```
badminton-highlight-indexer\backend\main.py:965:    (**DEFAULT-OFF**: ``get_compute_backend``
returns ``None`` for every non-``cloud_run`` target, so
badminton-highlight-indexer\backend\main.py:967:    from backend.compute import ComputeJobSpec,
get_compute_backend
badminton-highlight-indexer\backend\main.py:1013:        compute = get_compute_backend(cfg,
target_override="cloud_run")
badminton-highlight-indexer\backend\main.py:1015:        compute = get_compute_backend(
badminton-highlight-indexer\backend\compute\cloud_run.py:87:            "to resolve the job
path; set it on the control plane (see get_compute_backend)."
```

badminton-highlight-indexer\backend\compute\cloud_run.py:115:class
CloudRunCompute(ComputeBackend):

```
badminton-highlight-indexer\backend\eval\ablation.py:467:        audio_policy="tolerate-
missing",
badminton-highlight-indexer\backend\compute\__init__.py:3:COMPUTE_DECOUPLED_SERVING M6.
``get_compute_backend(config)`` is the seam: it returns a
badminton-highlight-indexer\backend\compute\__init__.py:22:    "get_compute_backend",
badminton-highlight-indexer\backend\compute\__init__.py:35:def get_compute_backend(
badminton-highlight-indexer\backend\compute\__init__.py:70:    return CloudRunCompute(
badminton-highlight-indexer\backend\compute\__init__.py:76:        policy=policy,
badminton-highlight-indexer\backend\infrastructure\database.py:1053:        "UPDATE jobs SET
policy=? WHERE id = ?",
badminton-highlight-indexer\backend\worker\__main__.py:30:from backend.compute import
ComputeJobSpec, get_compute_backend
badminton-highlight-indexer\backend\worker\__main__.py:200:    # DEFAULT-OFF:
get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall through to the
badminton-highlight-indexer\backend\worker\__main__.py:203:    compute =
get_compute_backend(config)
badminton-highlight-indexer\backend\eval\fusion_audio.py:80:        audio_policy="require",
badminton-highlight-indexer\backend\eval\fusion_compare.py:48:        audio_policy="omit",
```

11. user (2026-07-02T03:00:56.713Z)

```
badminton-highlight-indexer\scripts\verify_compute_path.py:83:    ap.add_argument("--timeout-
sec", type=float, default=1200)
badminton-highlight-indexer\docs\CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md:57:    --cpu 4 --memory 16Gi
--max-retries 0 --task-timeout 3600 `
badminton-highlight-indexer\deploy\gcp\nightly_regression\register_scheduler.sh:57:
--image="$IMAGE" --service-account="$SA" --max-retries=0 --task-timeout=900s \
badminton-highlight-indexer\backend\api\job_worker.py:37:#    - Cloud Run poll has a max_wait_sec
bound
badminton-highlight-indexer\docs\EXECUTION_POLICY.md:44:    max_wait_sec: float = 1800.0
# total wall budget across resumes before giving up
badminton-highlight-indexer\docs\EXECUTION_POLICY.md:64:5. give up when cumulative wait >
`max_wait_sec` ? `on_timeout` terminal action.
badminton-highlight-indexer\docs\EXECUTION_POLICY.md:106:- **Default WAIT_AND_RESUME, but
bounded** ? `max_wait_sec` guarantees forward progress / eventual give-up;
badminton-highlight-indexer\deploy\cloudrun\README.md:33:    --max-retries 0 --task-timeout 3600 \
badminton-highlight-indexer\deploy\cloudrun\README.md:63:    (`ExecutionPolicy.max_wait_sec`); the
worker CLI maps that to a clean **rc 4**, and the API path
badminton-highlight-indexer\backend\providers\gemini.py:169:        poll_every_n_sec=10.0,
max_wait_sec=1200.0
badminton-highlight-indexer\backend\providers\gemini.py:186:        f"[{log_prefix}]"
Processing exceeded {process_policy.max_wait_sec:.0f}s ? giving up."
badminton-highlight-indexer\backend\providers\gemini.py:190:        f"processing timed
out after {process_policy.max_wait_sec:.0f}s"
badminton-highlight-indexer\tests\test_api_cloud_dispatch.py:27:_FAST =
ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
badminton-highlight-indexer\tests\test_execution_policy.py:175:    pol =
ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1000.0)
badminton-highlight-indexer\tests\test_execution_policy.py:192:    pol =
ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=30.0)
badminton-highlight-indexer\tests\test_execution_policy.py:213:    pol =
ExecutionPolicy(poll_every_n_sec=5.0, max_wait_sec=100.0)
```

```

badminton-highlight-indexer\tests\test_compute_backend.py:21:_FAST =
ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
badminton-highlight-indexer\tests\test_compute_backend.py:258:         poll_every_n_sec=0.0,
op_timeout_sec=5.0, max_wait_sec=5.0
badminton-highlight-indexer\backend\infrastructure\execution_policy.py:8:     ``max_wait_sec``
deadline).
badminton-highlight-indexer\backend\infrastructure\execution_policy.py:42:     """An op hung past
``op_timeout_sec`` or never became ready within ``max_wait_sec``."""
badminton-highlight-indexer\backend\infrastructure\execution_policy.py:54:     max_wait_sec:
float = 1800.0 # total wall budget for a poll loop
badminton-highlight-indexer\backend\infrastructure\execution_policy.py:162:     ``PolicyTimeout``
after ``max_wait_sec``. ``check`` returns ``None`` while the op is still in progress.
badminton-highlight-indexer\backend\infrastructure\execution_policy.py:170:         "%s: waiting
(poll_every=%0.0fs, max_wait=%0.0fs)",
badminton-highlight-indexer\backend\infrastructure\execution_policy.py:173:
policy.max_wait_sec,
badminton-highlight-indexer\backend\infrastructure\execution_policy.py:183:         if elapsed >=
policy.max_wait_sec:
badminton-highlight-indexer\backend\infrastructure\execution_policy.py:185:         "%s:
still not ready after %0.0fs (max_wait) ? giving up", label, elapsed
badminton-highlight-indexer\backend\infrastructure\execution_policy.py:188:
f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"

```

12. user (2026-07-02T03:01:02.046Z)

```

1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5     ``None``
6     otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
7     identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
8     changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
9
10    """
11
12    from __future__ import annotations
13
14    import os
15    from typing import Any, Optional
16
17    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
18    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
19
20    __all__ = [
21        "ComputeBackend",
22        "ComputeJobSpec",
23        "ComputeResult",
24        "get_compute_backend",
25    ]
26
27    def _storage_dict(config: Any) -> dict:
28        storage = getattr(config, "storage", None)
29        if storage is None:
30            return {}
31        if hasattr(storage, "model_dump"):
32            return storage.model_dump() # type: ignore[no-any-return]
33        return dict(storage) if isinstance(storage, dict) else {}
34
35    def get_compute_backend(
36        config: Any,

```

```

37     *,
38     run_client: Optional[Any] = None,
39     policy: ExecutionPolicy = DEFAULT_POLICY,
40     token: Optional[str] = None,
41     target_override: Optional[str] = None,
42 ) -> Optional[ComputeBackend]:
43     """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
44     in-process (the default-OFF path).
45
46     Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
47     Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
48     / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
49     pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
50
51     ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
52     per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
53     server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
54     """
55     target = (
56         target_override
57         if target_override is not None
58         else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
59     )
60     if target != "cloud_run":
61         return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
62
63     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
64     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
65
66     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
67     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
68     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
69     client = run_client if run_client is not None else GoogleCloudRunJobClient()
70     return CloudRunCompute(
71         run_client=client,
72         job_name=job_name,
73         region=region,
74         project=project,
75         storage_config=_storage_dict(config),
76         policy=policy,
77         token=token,
78     )
79

```

13. user (2026-07-02T03:01:03.023Z)

```

20     # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21     REGION=asia-southeast1
22     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
23     # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
24     gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
25     ```
26     *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
D17.)*
27

```

```

28     ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
29     ```bash
30     gcloud run jobs create rally-worker \
31         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
32         --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
33         --max-retries 0 --task-timeout 3600 \
34         --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
35     # (stage the weights into the container at start, or fetch from gs:// in an init step ?
36     WASB-C3.)
37     ```
38     ## 3. Wire the dispatcher (control plane)
39     The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
40     serving` ?
41     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
42     from env:
43     ```bash
44     export DEPLOYMENT_PROFILE=cloud-serving
45     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
46     GOOGLE_CLOUD_PROJECT=$PROJECT
47     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
48     instead of
49     # running in-process; the container runs the SAME worker INLINE (compute_target forced
50     local_gpu).
51     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
52     gs://<bucket> --segmenter wasb_hybrid
53     ```
54     ## 4. M7 acceptance (the owner runlog)
55     Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
56     `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
57     commands,
58     rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
59     (`ffmpeg -f null -`
60     exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
61     ## Gotchas
62     - The dispatched container **must run inline** ? `CloudRunCompute` forces
63     `compute_target=local_gpu`
64     + `indexing.detector_impl=native` in the per-execution args so the Job never re-
65     dispatches itself.
66     - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
67     Run API status
68     ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
69     control-plane restart.
70     - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
71     carrying the
72     real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
73     hang on a bad
74     video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
75     time out
76     (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
77     the API path
78     surfaces it as a `cloud dispatch error` job failure (not a traceback).
79     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
80     libx264.
81     - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
82     `GoogleCloudRunJobClient.run_job`
83     fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
84     `job_path`;
85     an unset project would otherwise build a bogus `projects/None/...` name). Export it
86     (step 3) before
87     dispatching. The guard runs before the `google-cloud-run` import, so it is exercised
88     in CI even
89     though the SDK isn't a CI/test dependency.

```

14. assistant (2026-07-02T03:01:46.436Z)

All elements of the claim verified against the current tree. One material correction to the failure framing: billing is bounded, not indefinite.

15. user (2026-07-02T03:02:02.101Z)

Structured output provided successfully